

JAVA

Complete Concepts & Interview Guide

15 dense revision pages covering Core Java, OOP, JVM, exceptions, collections, generics, strings, multithreading, Java 8+, Streams, I/O, JDBC, memory and interview concepts.

Use this as a concept reference: understand what each feature does, why it exists, and where you would use it in a backend application.

1. Java Basics & Execution

Java and its purpose

- Java is a high-level, class-based, object-oriented language. Java source is compiled into bytecode, which a JVM can execute.
- Java is strongly and statically typed. Types are checked at compile time, reducing many classes of runtime errors.

JDK, JRE, JVM

- JDK provides development tools such as javac and the runtime needed to develop/run applications. JVM is the engine that loads and executes bytecode. The term JRE historically described JVM plus runtime libraries.
- Interview shortcut: JDK = develop, JVM = execute bytecode.

Execution flow

- .java → javac → .class bytecode → class loading/linking → JVM execution → JIT may compile hot code to native machine code.
- Java is portable because bytecode targets the JVM rather than one operating system's CPU instruction set.

Main method

```
public static void main(String[] args) {  
    System.out.println("Hello Java");  
}
```

Important concepts

- Class = blueprint/type. Object = runtime instance. Reference = variable used to access an object. null = no object reference.
- Java has automatic garbage collection and does not expose normal C-style pointer arithmetic.

2. Data Types, Variables & Operators

Primitive types

- Eight primitives: byte, short, int, long, float, double, char, boolean.
- Numeric literals may need suffixes such as L for long and f for float.

Reference types

- Classes, arrays, interfaces, enums and records are reference types. A reference can be null.
- Local variables do not receive automatic default values; initialize them before reading. Instance/static fields do receive defaults.

Variables

- Local variables belong to a method/block. Instance fields belong to an object. static fields belong to the class. final prevents reassignment after initialization.

Operators

- Arithmetic: + - * / %. Relational: < > <= >= == !=. Logical: && || !. Assignment: = += -=. Ternary: condition ? a : b.
- && and || short-circuit. This matters when the second expression could have side effects or throw an exception.

Casting

- Widening such as int → long is generally automatic. Narrowing can lose information and needs a cast.

```
int a = 10;  
long b = a;  
int c = (int)b;
```

3. Classes, Objects, Constructors & static

Classes and objects

- A class defines state and behavior. new creates/initializes an object and produces a reference.

```
class User {  
    String name;  
    void greet() { System.out.println(name); }  
}  
User u = new User();
```

Constructors

- Constructors initialize objects and have the class name with no return type. If no constructor is declared, the compiler can provide a no-argument constructor.
- Constructor overloading allows multiple constructors with different parameter lists.

this

- this refers to the current object. It resolves naming conflicts and can call another constructor using this(...).

```
User(String name) {  
    this.name = name;  
}
```

static

- Static members belong to the class. A static method has no implicit this and cannot directly access instance fields.
- Use static for class-level state/behavior, but avoid unnecessary mutable static state.

Access control

- private = class only; package-private = same package; protected = package plus permitted subclass access; public = broadly accessible.
- Encapsulation usually starts with private fields and controlled public methods.

4. OOP: Encapsulation, Inheritance & Composition

Encapsulation

- Encapsulation keeps data and the operations that manage it together while controlling access. It helps protect invariants.
- A private balance field with deposit/withdraw methods is safer than exposing balance for arbitrary modification.

Inheritance

- extends creates a subclass relationship. Java supports single inheritance for classes.

```
class Animal { void sound() {} }  
class Dog extends Animal { void bark() {} }
```

super

- super(...) calls a parent constructor and must be the first constructor statement. super.method() can call an overridden parent implementation.

is-a vs has-a

- Inheritance models is-a. Composition models has-a and is often more flexible. Prefer composition when behavior should be assembled rather than rigidly inherited.

Object lifecycle idea

- A reference can be reassigned while the object remains alive if another live reference exists. An object becomes eligible for garbage collection when it is unreachable from GC roots.

5. Polymorphism, Abstraction & Interfaces

Polymorphism

- Overloading is compile-time selection based on method signature. Overriding is runtime dispatch based on the actual object.

```
Animal a = new Dog();  
a.sound(); // overridden Dog method
```

Overloading

- Same method name, different parameter list. Return type alone cannot overload a method.

```
void print(int x) {}  
void print(String x) {}
```

Overriding

- A subclass provides a compatible implementation of an inherited method. Use @Override so the compiler checks the signature.

Abstract class

- Cannot be instantiated directly. It can contain fields, constructors, concrete methods and abstract methods.
- A concrete child must implement inherited abstract methods or itself be abstract.

Interface

- An interface defines a contract. A class can implement multiple interfaces. Modern interfaces may contain abstract, default, static and private methods.
- Interface fields are implicitly public static final.

6. Object, equals, hashCode & toString

Object class

- Object is the root superclass for ordinary Java classes. Important methods include equals, hashCode, toString, getClass and wait/notify-related methods.

== vs equals

- For primitives, == compares values. For references, == checks whether references point to the same object.
- equals is for logical equality and should be overridden when object value equality matters.

hashCode contract

- If a.equals(b) is true, a.hashCode() must equal b.hashCode(). Unequal objects may still have the same hash code because collisions are possible.
- If equals is overridden, hashCode should normally be overridden too.

toString

- toString gives a readable representation useful for logs/debugging.

```
@Override
public String toString() {
    return "User{" + name + "}";
}
```

Common interview trap

- HashMap uses hash information to locate candidate entries and equality checks to distinguish matching keys. A hash collision does not mean two keys are equal.

7. Strings, Arrays & Wrapper Classes

String

- String is immutable. Operations that appear to modify it create another String.
- String literals can be interned in the string pool, allowing identical literals to share storage.

String comparison

- Use equals for content. == checks reference identity for objects.

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
```

StringBuilder

- StringBuilder is mutable and is useful for repeated concatenation. StringBuffer is synchronized and generally chosen only when that synchronization is required.
- Know methods such as append, insert, delete and reverse.

Arrays

- Arrays have fixed length and zero-based indexing. java.util.Arrays provides sort, binarySearch, copyOf, equals and toString.

```
int[] a = {3,1,2};
Arrays.sort(a);
```

Wrappers and boxing

- Integer, Long, Double, Character and Boolean wrap primitives as objects. Autoboxing/unboxing happens automatically.
- Beware null unboxing: Integer x = null; int y = x; causes NullPointerException.

8. Collections Framework

Interfaces

- List is ordered and allows duplicates. Set models uniqueness. Queue models processing order. Map stores key-value pairs and is separate from Collection.

ArrayList

- Resizable array with fast indexed access. Insertions/removals in the middle may shift elements. Often the default List choice.

LinkedList

- Linked structure implementing List and Deque. It can support efficient operations at ends, but ArrayList often performs better for general workloads because of locality.

Set implementations

- HashSet: hashing, no guaranteed order. LinkedHashSet: predictable insertion order. TreeSet: sorted order and navigation operations.

Queue/Deque

- ArrayDeque is useful for stack and queue behavior. PriorityQueue orders according to priority. Avoid legacy Stack when Deque is a better fit.

Choosing

- Need indexed access → ArrayList. Need uniqueness → Set. Need key-value lookup → Map. Need FIFO/LIFO → Queue/Deque. Need sorted keys/elements → TreeMap/TreeSet.

9. Map, Hashing, Comparable & Comparator

HashMap

- Stores key-value pairs and permits one null key. Average lookup is near $O(1)$ with good hashing. Correct equals/hashCode behavior is essential for keys.
- Do not mutate fields used by hashCode/equals while an object is acting as a key.

LinkedHashMap

- Maintains predictable iteration order, commonly insertion order. Access-order mode can support LRU-style designs.

TreeMap

- Maintains sorted keys and supports navigation such as lowerKey, floorKey, ceilingKey and higherKey.

Comparable

- Defines natural ordering inside the class through compareTo.

```
class Employee implements Comparable<Employee> {  
    public int compareTo(Employee e) {  
        return this.name.compareTo(e.name);  
    }  
}
```

Comparator

- Defines external/custom ordering. Multiple Comparator strategies can exist.

```
employees.sort(Comparator.comparing(Employee::getSalary));
```

Complexity reminder

- Hash-based collections usually give expected constant-time lookup; tree-based collections are typically $O(\log n)$. Always consider ordering, memory and workload rather than memorizing only complexity.

10. Generics & Type Safety

Why generics

- Generics provide compile-time type safety and reduce casts.

```
List<String> names = new ArrayList<>();
names.add("Java");
String x = names.get(0);
```

Generic class

```
class Box<T> {
    T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
}
```

Wildcards

- ? means unknown type. ? extends T is generally for reading values as T. ? super T is generally for consuming/inserting T values.
- PECS = Producer Extends, Consumer Super.

Invariance

- List<String> is not a subtype of List<Object>, even though String is a subtype of Object. This prevents unsafe insertion of unrelated objects.

Type erasure

- Java generics are largely implemented through type erasure. This is why you cannot normally create new T(), and why primitives such as int cannot be direct generic type arguments; use Integer.

Diamond operator

- The <> syntax lets the compiler infer generic type arguments on the right side.

```
Map<String, Integer> scores = new HashMap<>();
```

11. Exceptions & Resource Handling

Hierarchy

- Throwable is the root. Error generally represents serious JVM/system problems. Exception represents conditions applications may handle. RuntimeException is unchecked.

Checked vs unchecked

- Checked exceptions must generally be caught or declared. RuntimeException subclasses are unchecked.
- IOException and SQLException are common checked exceptions. NullPointerException and IllegalArgumentException are unchecked.

try/catch/finally

- try contains risky code; catch handles matching failures; finally is commonly used for cleanup.

```
try {
    risky();
} catch (IOException e) {
    log(e);
} finally {
    cleanup();
}
```

throw vs throws

- throw explicitly creates/throws an exception. throws declares exceptions that may propagate from a method.

try-with-resources

- Use with AutoCloseable resources such as files and JDBC objects. Java closes them automatically.

```
try (BufferedReader br = Files.newBufferedReader(path)) {
    System.out.println(br.readLine());
}
```

Best practices

- Catch exceptions you can meaningfully handle. Do not silently swallow exceptions. Preserve useful context and avoid exposing sensitive stack traces to end users.

12. Multithreading & Concurrency

Threads

- A thread is a path of execution. Calling `start()` begins concurrent execution; calling `run()` directly is only a normal method call.
- `Runnable` represents work without a returned result; `Callable` can return a result and throw checked exceptions.

Thread states

- `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING` and `TERMINATED` are the standard `Thread.State` values.

Race condition

- Occurs when multiple threads access shared mutable state and the result depends on timing.

synchronized

- Provides mutual exclusion and memory-visibility guarantees for a monitor.

```
private synchronized void increment() {  
    count++;  
}
```

volatile

- Ensures visibility of writes/reads across threads and affects ordering, but does not make compound operations such as `count++` atomic.

ExecutorService

- Thread pools are preferred for managing many tasks instead of manually creating threads.

```
ExecutorService pool = Executors.newFixedThreadPool(4);  
pool.submit(() -> work());  
pool.shutdown();
```

13. Concurrency Utilities & CompletableFuture

Atomic classes

- AtomicInteger and related classes provide atomic operations for suitable shared counters/state.

```
AtomicInteger count = new AtomicInteger();  
count.incrementAndGet();
```

Locks

- ReentrantLock provides explicit locking and features such as tryLock. Unlock in finally after successful lock acquisition.

Concurrent collections

- ConcurrentHashMap supports concurrent map access. BlockingQueue is useful for producer-consumer designs.

Deadlock

- Deadlock occurs when threads wait forever for resources held by one another. Reduce risk with consistent lock ordering, short critical sections and timeouts where appropriate.

CompletableFuture

- Represents an asynchronous computation and supports composition.

```
CompletableFuture.supplyAsync(() -> fetch())  
    .thenApply(data -> transform(data))  
    .thenAccept(System.out::println);
```

Executor mental model

- Submit tasks to an executor → worker executes them → Future/CompletableFuture represents the result → shutdown the executor when its lifecycle ends.

14. Java 8+, Streams, I/O, JDBC & Modern Java

Lambdas and functional interfaces

- A lambda is a concise representation of behavior. Functional interfaces have one abstract method. Common ones: Predicate, Function, Consumer and Supplier.

```
names.forEach(n -> System.out.println(n));
```

Streams

- Streams are pipelines over data sources. Intermediate operations are lazy; terminal operations trigger processing.
- filter selects, map transforms, flatMap flattens, sorted orders, reduce combines and collect/toList produces results.

```
List<String> out = names.stream()  
    .filter(n -> n.length() > 3)  
    .map(String::toUpperCase)  
    .toList();
```

Optional

- Optional represents a possibly absent value and can make absence explicit. It is useful in APIs but is not a universal replacement for null.

I/O and NIO

- InputStream/OutputStream handle bytes. Reader/Writer handle characters. Path and Files provide modern file APIs.
- Use try-with-resources for closeable resources.

JDBC

- Typical flow: Connection → PreparedStatement → execute → ResultSet → close. Use parameterized PreparedStatement rather than concatenating untrusted SQL.

```
String sql = "SELECT * FROM users WHERE id = ?";  
PreparedStatement ps = con.prepareStatement(sql);  
ps.setInt(1, id);
```

Modern Java

- Records provide concise data carriers. Sealed classes restrict inheritance. Pattern matching and switch expressions can simplify control flow. Exact features depend on the JDK version.

15. JVM, Garbage Collection & Interview Revision

Memory concepts

- Heap stores objects managed by the JVM. Each thread has execution-stack frames. Class metadata is maintained in JVM runtime structures such as metaspace in common HotSpot implementations.
- Do not treat simplified stack/heap diagrams as absolute rules for every JVM optimization.

Garbage collection

- An object is eligible for GC when it is unreachable from live GC roots. Calling `System.gc()` is only a request/hint, not a guarantee.
- GC reclaims unreachable objects; it does not simply delete objects because a variable goes out of scope if other references remain.

JIT and class loading

- Class loaders load classes. The JVM verifies, links and initializes classes as required. JIT compilation can optimize frequently executed bytecode into native code at runtime.

Top interview questions

- JDK vs JRE vs JVM? Explain the four OOP pillars. Overloading vs overriding? `==` vs `equals`? `equals`/`hashCode` contract? Why is `String` immutable? `ArrayList` vs `LinkedList`? `HashMap` vs `TreeMap`? `Comparable` vs `Comparator`? `Checked` vs `unchecked` exception? `throw` vs `throws`? `final`/`finally`/`finalize`? `synchronized` vs `volatile`? `Thread` vs `Runnable` vs `ExecutorService`? `Stream` vs `Collection`? `map` vs `flatMap`? What makes an object eligible for GC?

Backend connection

- In a Java backend, a controller receives a request, a service applies business logic, a repository/DAO accesses data, exceptions are handled centrally, and DTOs/models carry data. These layers use OOP, interfaces, collections, generics, exceptions, streams and concurrency together.

Final revision rule

- Do not memorize definitions only. For every concept, know: what it is, why it exists, a small code example, one advantage, one limitation, and one situation where you would use it.